



Visualizing Data with Daltoolbox

Choosing the right chart can make all the difference when communicating data. This section explores four powerful visualization types — **bar charts, lollipop plots, pie charts, and radar diagrams** — each offering a unique lens through which categorical differences and proportions come to life. Together, they form a versatile toolkit for turning raw data into clear, compelling insights.

R · DALTOOLBOX

GGPLOT2

Eduardo Ogasawara

eduardo.ogasawara@cefet-rj.br

<https://eic.cefet-rj.br/~eogasawara>

Navigating the Collection

The visualization examples are organized around **analytical questions** rather than function names — making it easy to find the right chart for what you need to communicate. Each section builds on the previous, moving from simple comparisons to more complex temporal and export workflows.

01

Category Comparison

Compare groups using bar charts, lollipop, pie, and radar — the foundation of most data stories

02

Distribution Analysis

Reveal the shape of your data through histograms, density curves, and boxplots

03

Relationships

Explore how variables interact using scatter plots and point-based visuals

04

Time Series

Track change over time with line charts, basic trends, and forecasting models

05

Export

Save publication-ready outputs in PDF, JPEG, or PNG using `ggsave`

Basic Graphics

Understanding how your data is distributed is the foundation of any meaningful analysis. This deck walks you through **17 practical examples** of data visualization in R — using **ggplot2** and **daltoolbox** — covering histograms, kernel density plots, and boxplots to reveal patterns, dispersion, and asymmetry hidden within your datasets. Whether you're exploring a new dataset or communicating insights to others, these visualizations will sharpen how you see and tell the story of your data.

R · DALTOOLBOX

GGPLOT2

Graphic Environment Setup

Before diving into the visualizations, every example in this collection shares a **common foundation**: the `daltoolbox` package for data handling, a consistent color palette via `RColorBrewer`, and a unified font size set through `theme()`. Define this once — and all 17 examples will build seamlessly on top of it.

```
# installation (if needed)
# install.packages("daltoolbox")

library(daltoolbox)
library(RColorBrewer)
library(ggplot2)

# default color palette
colors <- brewer.pal(4, 'Set1')

# font size for all charts
font <- theme(text = element_text(size=16))
```



Start here. This setup block is the single shared foundation across all 17 examples in the collection. Run it once at the top of your script, and every visualization that follows will work out of the box — no repeated configuration needed.

Core Didactic Principle

With the environment ready, every example in this collection is guided by a single underlying philosophy:

Treat the chart as an analytical tool, not decoration. Pay attention to the question the chart answers and how the input data should be organized before plotting.

This philosophy unfolds as a three-step progression — from intent to structure to output — applied consistently across all 17 examples:



Define the Question

Before choosing a chart, clearly define what needs to be communicated.



Organize the Data

Visualization starts with the correct organization of the input data.



Choose the Visual

Select the chart type that best answers the defined analytical question.

Bar Chart – Theory

A bar chart represents categorical data with rectangular bars whose heights (or lengths) are proportional to the values they represent.

Use it for comparing a single metric across discrete, unordered or ordered categories. It is ideal when the number of categories is small to moderate ($\leq \sim 15$).

Key Design Decisions

Orientation

Consider vertical vs. horizontal orientation; horizontal is preferred for long category names to prevent labels from overlapping.

Sorting

Sort bars by value (ascending or descending) to aid comparison and quickly identify high/low performers.

Aesthetics

Avoid 3D effects and unnecessary decoration, which can distort perception and distract from the data.

Strengths

- Easy to read and universally understood.
- Effective for magnitude comparison between categories.

Limitations

- Not ideal for showing distributions or trends over time.
- Can appear cluttered with too many categories, reducing readability.

Wilke, C. O. (2019). *Fundamentals of Data Visualization*. O'Reilly Media. Chapter 6: Visualizing amounts. Available at: <https://clauswilke.com/dataviz/>

Bar Chart

Applying the core principle — question first, then data, then visual — the bar chart is a natural starting point: it answers *"which group has the highest value?"* by comparing aggregated metrics across discrete categories.

```
library(daltoolbox)
library(RColorBrewer)
library(ggplot2)
library(dplyr)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Aggregate mean by species
data <- iris |>
  group_by(Species) |>
  summarize(Sepal.Length = mean(Sepal.Length))
```

```
# Simple bar
grf <- plot_bar(data, colors=colors[1]) + font
plot(grf)

# Horizontal version
grf <- grf + coord_flip()
plot(grf)

# One color per species
grf <- plot_bar(data, colors=colors[1:3]) +
  font
plot(grf)
```

Step by step

- Install and load `daltoolbox`, `ggplot2` and `RColorBrewer`
- Define the palette with `brewer.pal()` and font size with `theme()`
- Aggregate the data with `group_by()` + `summarize()`
- Call `plot_bar()` passing the data and colors
- Use `coord_flip()` to rotate the bars horizontally
- Pass `colors[1:3]` to color each category differently

❏ **Key concept:** The bar chart is the clearest embodiment of the analytical approach — once the question is defined and the data is organized by category, `plot_bar()` translates those aggregated values (means, counts, totals) directly into a visual comparison. The chart does not decorate; it answers: *"which group has the highest value?"*

Uncertainty Visualization – Error Bars

Error bars are graphical representations of the variability or uncertainty of a plotted quantity (e.g., standard deviation, standard error, confidence intervals). They provide crucial statistical context, helping to assess the reliability of comparisons within bar charts.



Standard Deviation (SD)

Shows the spread of individual data points around the mean.



Standard Error (SE)

Shows the precision of the sample mean estimate.



Confidence Interval (CI)

Shows the range within which the true population mean likely falls (e.g., 95% CI).

Use SD for describing data spread; SE and CI for inferential comparisons. A key rule is to always label which type of error bar is shown — unlabeled error bars are misleading.

Strengths

- Adds statistical context to bar charts.
- Helps readers assess reliability of comparisons.

Limitations

- Can be misread if type is not specified.
- Overlapping bars don't always mean non-significance.

Cumming, G., Fidler, F., & Vaux, D. L. (2007). Error bars in experimental biology. *Journal of Cell Biology*, 177(1), 7–11. <https://doi.org/10.1083/jcb.200611141>

Bar Chart + Error Bars

The base bar chart shows *central values* — but how variable is each group? Adding error bars reveals the *spread* behind each mean.

```
library(daltoolbox)
library(ggplot2)
library(RColorBrewer)
library(dplyr)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Calculate mean and standard deviation
data <- iris |>
  group_by(Species) |>
  summarize(
    mean = mean(Sepal.Length),
    sd = sd(Sepal.Length)
  )
```

```
# Base bar chart
grf <- plot_bar(data, colors=colors[1],
  alpha=1) + font

# Add error bars
grf <- grf + geom_errorbar(
  aes(x=x, ymin=y-sd, ymax=y+sd),
  width=0.2,
  colour="darkred",
  alpha=0.8,
  size=1.1
)
plot(grf)
```

Step by step

- Calculate mean and sd in the same summarize()
- Create the base bar chart with plot_bar()
- Add geom_errorbar() to the grf object with +
- Set ymin = y - sd and ymax = y + sd
- Adjust width, colour, and size for aesthetics

📌 **Key concept:** This *extends* the previous bar chart by layering geom_errorbar() on top. While the bars still answer "*which group is highest?*", the error bars now also answer "*how reliable is that comparison?*" — adding uncertainty and variability (e.g., standard deviation) to the same visual.

Grouped & Stacked Bar Charts – Theory

Grouped and stacked bar charts are extensions of the basic bar chart that handle two or more measures per category, allowing for richer comparisons and compositional analysis.

Grouped (Side-by-Side) Bars

Each sub-group is placed next to the other within the same category. This layout is best for comparing individual values across sub-groups within each category.

Question answered: **"Is measure A bigger than measure B within each category?"**

Stacked Bars

Sub-groups are stacked on top of each other, where the total height represents the sum of all measures for that category. This is best for showing cumulative totals and proportional composition.

Question answered: **"What is the total, and how is it composed?"**

A variant, **100% stacked bars**, normalizes each bar to 100%, emphasizing proportions rather than absolute values. A key design rule for both is to use consistent colors per sub-group across all categories. Additionally, avoid stacking more than 4–5 sub-groups as it quickly becomes hard to read and interpret.

Cairo, A. (2016). *The Truthful Art: Data, Charts, and Maps for Communication*. New Riders. Chapter 5: Basic Principles of Visualization.

Grouped and Stacked Bar Charts

Building on single-metric bars, these variants handle **two or more measures per category** — choose the layout based on what you want to emphasize.

```
library(daltoolbox); library(ggplot2)
library(RColorBrewer); library(dplyr)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Two metrics per species
data <- iris |>
  group_by(Species) |>
  summarize(
    Sepal.Length = mean(Sepal.Length),
    Sepal.Width = mean(Sepal.Width)
  )
```

```
# Grouped bars (side-by-side)
grf <- plot_groupedbar(data,
  colors=colors[1:2]) + font
plot(grf)

# Stacked bars
grf <- plot_stackedbar(data,
  colors=colors[1:2]) + font
grf <- grf + theme(
  axis.text.x = element_text(angle=90,
    hjust=1)
)
plot(grf)
```

Step by step

- Aggregate **two metrics** in the same `summarize()`
- `plot_groupedbar()`: displays bars side by side per category
- `plot_stackedbar()`: stacks the metrics; total height = sum
- Rotate X-axis labels with `element_text(angle=90)` when needed
- Pass `colors[1:2]` to distinguish the two measures

❏ **Key concept:** Use *grouped* bars to **compare measures against each other** within each category (e.g., "Is Sepal.Length bigger than Sepal.Width?"). Use *stacked* bars to emphasize **cumulative totals and proportional composition** — where the combined height matters as much as the individual parts.

Lollipop Chart – Theory

The lollipop chart is a variation of the bar chart where each bar is replaced by a thin line (stem) topped with a dot (marker), resembling a lollipop. It provides an alternative visual representation for quantitative comparisons across categories.

Use a lollipop chart when you have many categories and traditional bars would create visual clutter. They are particularly effective when values are close together and the filled area of bars adds noise rather than clarity, or when you want to reduce the "ink-to-data ratio" for a cleaner aesthetic.

Lollipop Chart

Lighter, with less visual weight, making it better for dense comparisons and when the exact value is less critical than its position.

Bar Chart

Better when the filled area helps the audience quickly judge magnitude at a glance and when absolute comparisons are a primary goal.

Key design decisions for lollipop charts include sorting categories by value for improved readability, and using a consistent dot size across all markers. It is crucial to avoid connecting lines between dots, as this would imply continuity or a trend that does not exist in categorical data.

This chart type aligns with the concept of "data-ink ratio" proposed by Edward Tufte: maximize the proportion of ink used to convey data, while minimizing decorative or redundant ink. By replacing thick bars with thin lines and dots, lollipop charts achieve a higher data-ink ratio compared to traditional bar charts.

Tufte, E. R. (2001). *The Visual Display of Quantitative Information* (2nd ed.). Graphics Press. Chapter 4: Data-Ink and Graphical Redesign.

Lollipop: A Modern Alternative to Bar Charts

```
library(daltoolbox); library(ggplot2)
library(RColorBrewer); library(dplyr)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Aggregate mean by species
data <- iris |>
  group_by(Species) |>
  summarize(Sepal.Length =
    mean(Sepal.Length))
```

```
# Vertical lollipop
grf <- plot_lollipop(
  data,
  colors = colors[1],
  max_value_gap = 0.2
) + font
plot(grf)

# Horizontal lollipop
grf <- plot_lollipop(
  data,
  colors = colors[1],
  max_value_gap = 0.1
) + font + coord_flip()
plot(grf)
```

Step by step

- Use the same aggregated data as the bar chart
- Call `plot_lollipop()` with `max_value_gap` to control the space between the marker and the top
- Add `coord_flip()` for the horizontal version
- Visually lighter than bars: highlights the value with a marker and stem

📌 **Key concept:** The lollipop is a *visually lighter alternative* to the bar chart — prefer it when you have **many categories**, values are **close together**, or you want to **reduce ink and clutter**. Stick with bars when filled area is needed to judge magnitude at a glance.

Pie & Radar Charts – Theory

When visualizing categorical data, pie charts and radar charts offer distinct ways to represent proportions and multi-dimensional profiles, respectively. Each has specific strengths and weaknesses that dictate its appropriate use.

Pie Chart

What it is: a circular chart divided into slices, where each slice's angle/area is proportional to the quantity it represents

Best use case: showing part-to-whole proportions with a small number of categories (≤ 5)

Question answered: **"What share does each piece represent of the whole?"**

Limitations: humans are poor at comparing angles and areas; avoid when differences between slices are subtle; never use 3D pie charts (distorts perception)

Rule of thumb: if you can't tell the difference between slices, use a bar chart instead

Radar Chart (Spider/Web Chart)

What it is: a two-dimensional chart with multiple axes radiating from a central point, each representing a different variable; values are plotted on each axis and connected to form a polygon

Best use case: comparing multiple numeric variables simultaneously for a single entity or comparing profiles of a few entities

Question answered: **"How does this entity's profile look across all dimensions at once?"**

Limitations: hard to read with many variables (> 8); difficult to compare multiple entities; axes with different scales can be misleading

Few, S. (2007). Save the Pies for Dessert. *Perceptual Edge Visual Business Intelligence Newsletter*. Available at: https://www.perceptualedge.com/articles/visual_business_intelligence/save_the_pies_for_dessert.pdf

Pie & Radar: Specialized Charts for Specific Use Cases

While bar and lollipop charts handle most category comparisons, **pie** and **radar** charts each shine in distinct scenarios — proportional composition and multivariate profiling, respectively.

```
library(daltoolbox); library(ggplot2)
library(RColorBrewer); library(dplyr)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# --- Pizza ---
data <- iris |>
  group_by(Species) |>
  summarize(Sepal.Length = mean(Sepal.Length))

grf <- plot_pieplot(data, colors=colors[1:3]) + font
plot(grf)
```

```
# --- Radar ---
data_radar <- iris |>
  dplyr::select(-Species)
data_radar <- apply(data_radar, mean)
data_radar <- data.frame(
  name = names(data_radar),
  value = data_radar
) |> dplyr::arrange(name)

grf <- plot_radar(data_radar, colors=colors[1])
+ font
grf <- grf + ylim(0, NA)
plot(grf)
```

Step by step – Pie

- Use `plot_pieplot()` with few categories (≤ 5)
- Pass one color per slice via `colors[1:3]`
- Use sparingly: small angles are hard to compare

Step by step – Radar

- Calculate the mean of each numeric variable with `apply()`
- Format as `data.frame(name, value)` and sort
- Use `ylim(0, NA)` to fix the origin at zero

📌 **Key concept:** Choose *Pie* when you want to show **part-to-whole proportions** across a small number of categories (≤ 5) — it answers "what share does each piece represent?" Choose *Radar* when you need to **compare multiple numeric variables simultaneously** on a single entity — it answers "how does this profile look across all dimensions at once?" Avoid pie when differences are subtle; avoid radar when variables have very different scales.

Summary: Visual Toolkit

Each chart type answers a **specific analytical question** — the same five categories that guided your journey through this collection. Choose your chart based on what you need to communicate, not aesthetics.

Comparison

Bar, Grouped Bar, Stacked Bar, Lollipop, Pie, Radar — *"Which group stands out, and by how much?"*

Distribution

Histogram, Density, Boxplot — *"Where does the data cluster, and how spread out is it?"*

Relationships

Scatter, Points — *"Do two variables move together, and how strongly?"*

Time Series

Lines, Basic TS, Forecast — *"How does a value change over time, and what comes next?"*

Export

`ggsave()` for crisp PDF/PNG output; `jpeg()` + `dev.off()` for pixel-level JPEG control — share your insights anywhere.

📌 Start with the question → pick the category → select the chart. This workflow is the heart of the collection.

References & Additional Sources

Academic references and online resources to explore the DAL Toolbox and Experiment Lines further.

Online Resources



CRAN Package – daltoolbox

cran.r-project.org/web/packages/daltoolbox

DAL Toolbox – GitHub

github.com/cefet-rj-dal/daltoolbox

GitHub Repository

github.com/cefet-rj-dal/daltoolbox

Source code, issues, and contributions.

Key Academic References

- Ogasawara, E., Sa, A., Castro, A., Santos, C., Salles, D., Lima, J., Tavares, L., Salles, R. & Saidy, V. (2026). daltoolbox: Leveraging Experiment Lines to Data Analytics. R package version 1.3.767. *CRAN*.
- Ogasawara, E., Paulino, C., Murta, L., Werner, C. & Mattoso, M. (2009). Experiment Line: Software Reuse in Scientific Workflows. *SSDBM 2009, LNCS 5566*, pp. 264–272. *Springer*.
- Ogasawara, E., Castro, A., Gea, C., Borges, H., Carvalho, D., Santos, J., Bezerra, E. & Coutinho, R. (2023). Leveraging Experiment Lines to Data Analytics. *DAL/CEFET-RJ*.
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *IJCAI*, 14(2), 1137–1145.
- Guyon, I. & Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning. 2nd ed. *Springer*.